CLIMB

When to Call the Team

Save tokens on multi-agent LLM pipelines, with a guarantee on accuracy

Jianjun Gao, Jinxin Liu, Dongyun Lin, Hongyun Cai, Zheng Ma

EXECUTIVE SUMMARY

Call the team only when you must, with a guarantee

Problem	Agent teams are more accurate, but use up to **6.3×** the tokens of a single agent and break right answers on up to **9%** of tasks.
Idea	Try cheap answers first. Call the team only when a cheap answer looks unreliable (**confidence**, **agreement**).
Guarantee	You set the accuracy you can lose. The **data sets the threshold**: a **betting test** certifies the cheapest one that keeps it, **90%** of the time.
Result	**38–65% fewer tokens** on 3 models × 4 benchmarks. The promise broke in **≤ 2%** of splits (10% allowed).
For Amex	Sits in front of any multi-agent workflow. Lessons for designing complex agent systems.

BACKGROUND

What is an agent team?

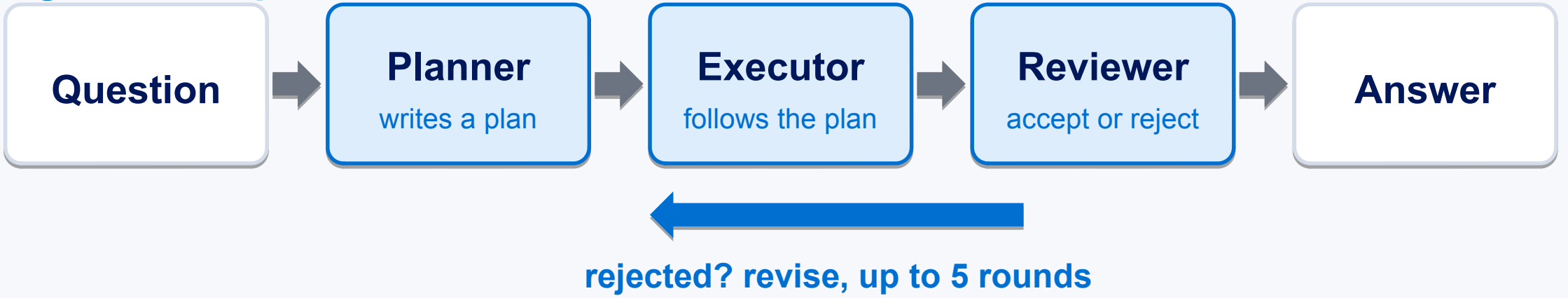
Single agent



+2 to +21

points over a direct answer. Best on 12 / 12 settings.

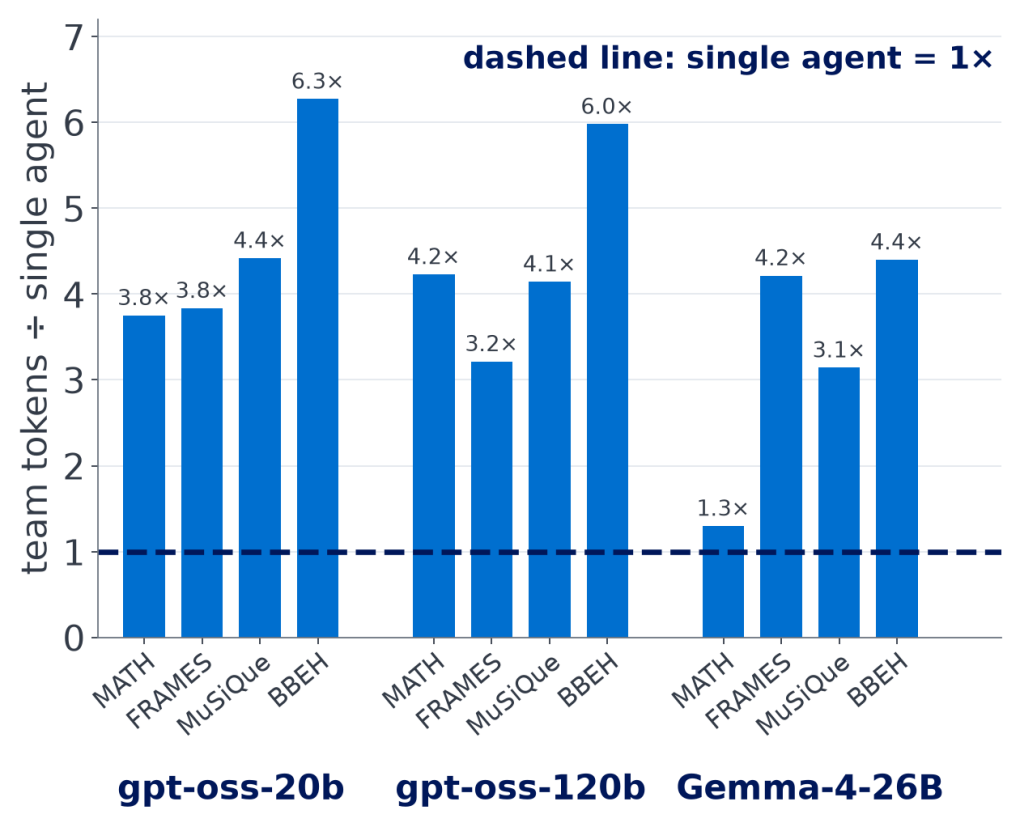
Agent team: plan, execute, review



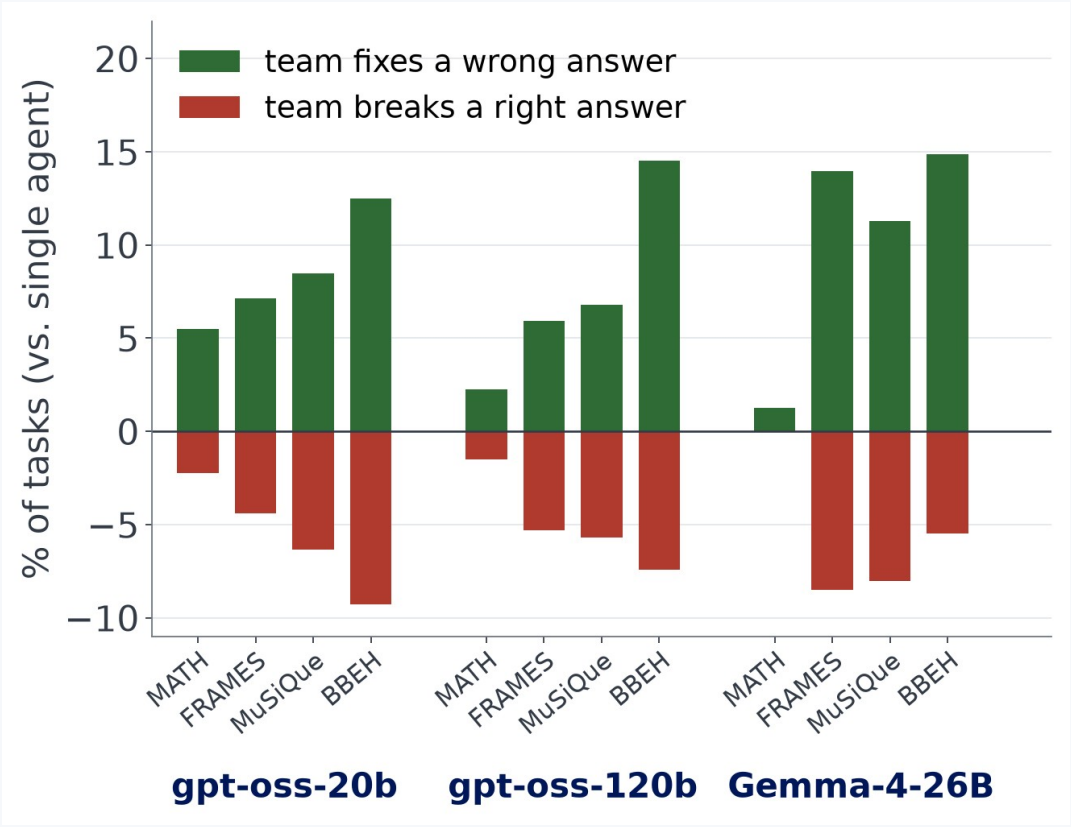
THE CATCH

Teams are costly, and sometimes harmful

Cost: up to 6.3× a single agent



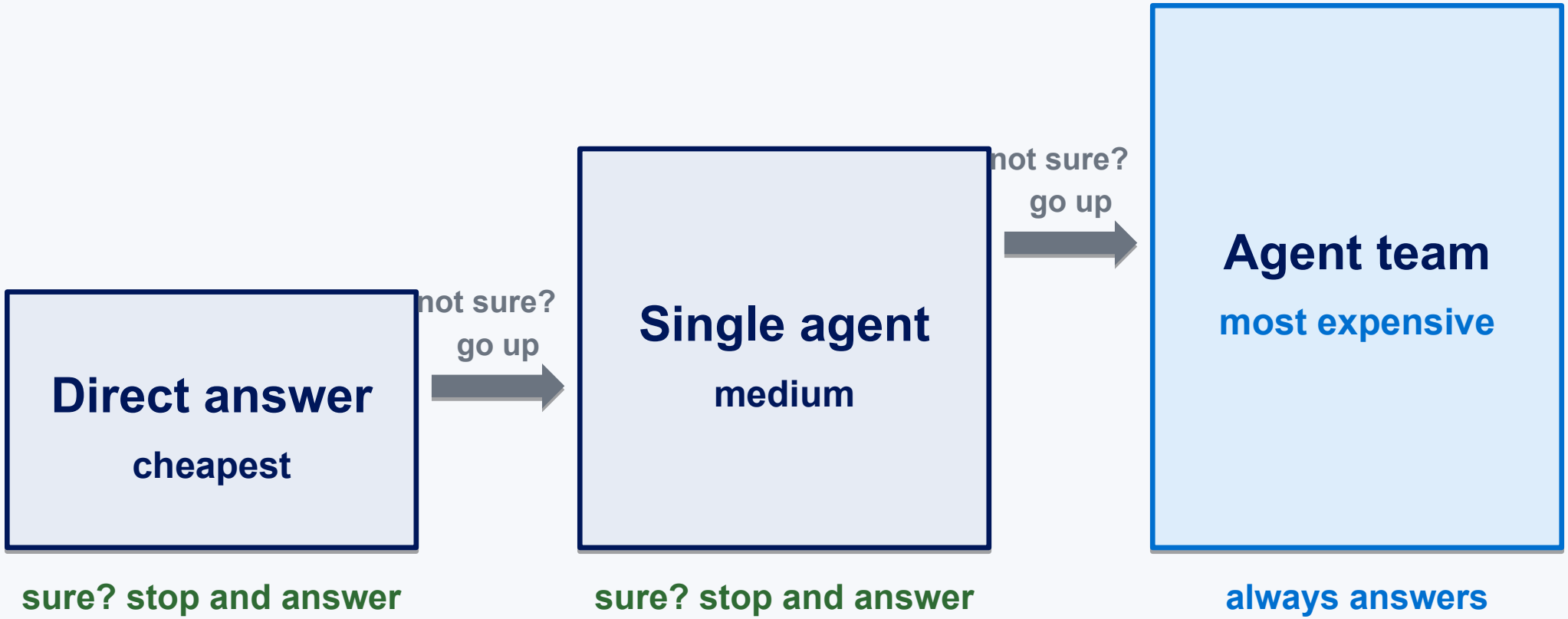
Harm: up to 9% of tasks broken



THE IDEA

Climb a ladder: call the team only when needed

Stop if score > threshold. The key question: which threshold?



STEP 1 · THE SCORE

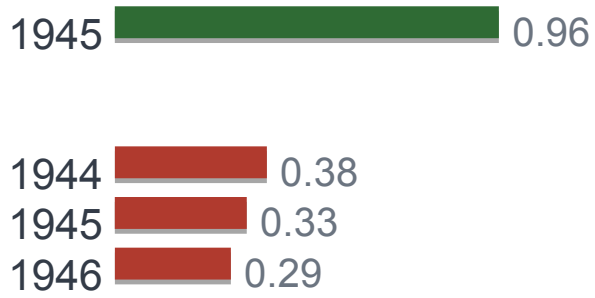
Why confidence, and why agreement?

A good signal tells right from wrong, and costs far less than the team.

Confidence

Is the model sure of its answer?

Example: “When did World War II end?”



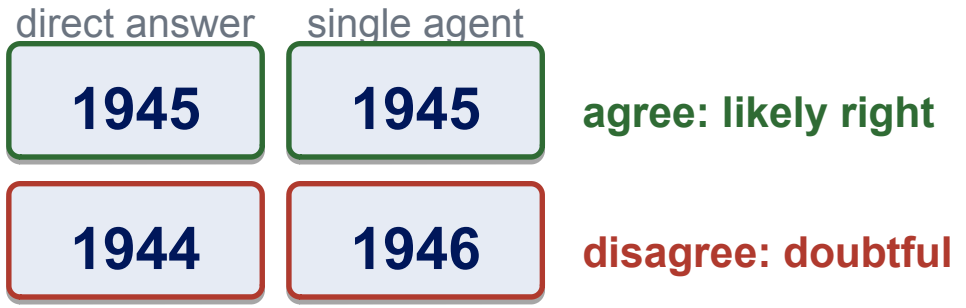
A model that knows tends to put its probability on **one** answer. An unsure one spreads it.

Cost: free, it comes with the answer.

Agreement

Do two cheap answers match?

Example: “When did World War II end?”



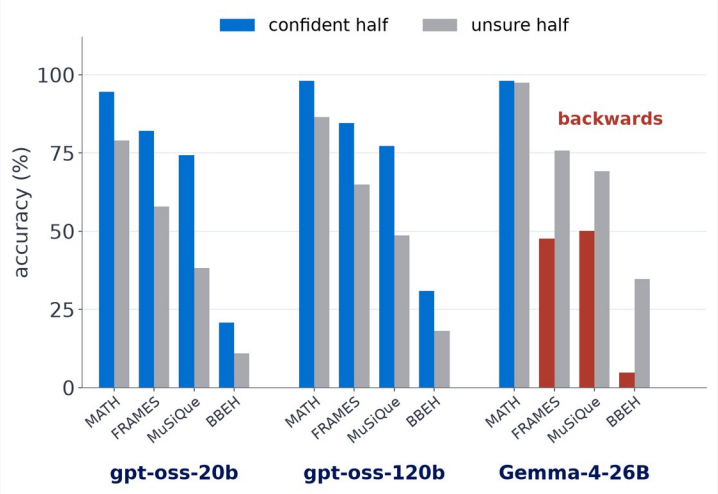
One right answer, many wrong ones: two tries **rarely** match on a wrong one.

Cost: one more cheap call, no log-probs.

STEP 1 · THE SCORE

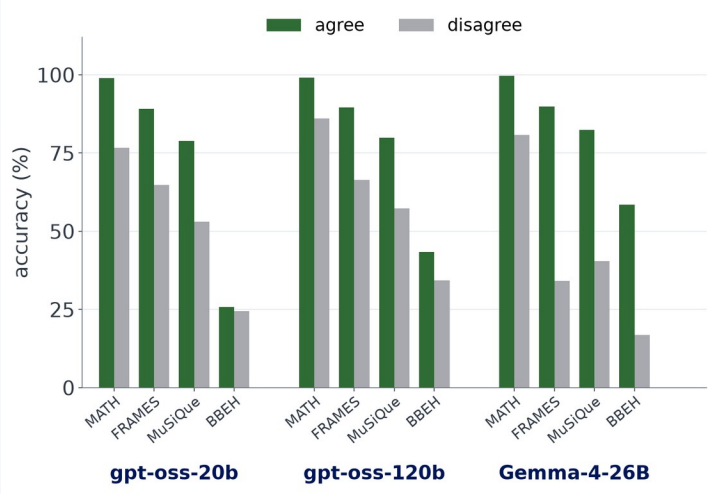
Do they work? Mostly, and they cover each other

Confidence: direct answer accuracy



Mostly helps. **Backwards on 3 (Gemma).**

Agreement: single agent accuracy



Mostly helps, **including those 3.**

When confidence is backwards, CLIMB still holds:

1 Agreement first

Option B ranks by agreement. Confidence only breaks ties.

2 The check catches it

A backwards score fails the test early. It is not deployed.

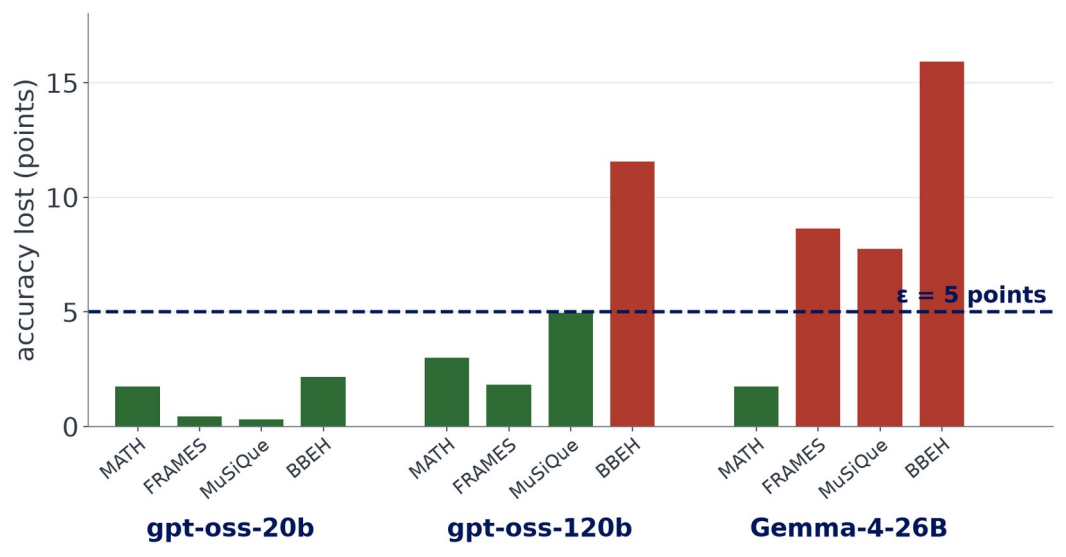
3 Worst case: the team

Less saving, no lost accuracy. Gemma: 69–96% of tokens.

STEP 2 · SETTING THE THRESHOLD

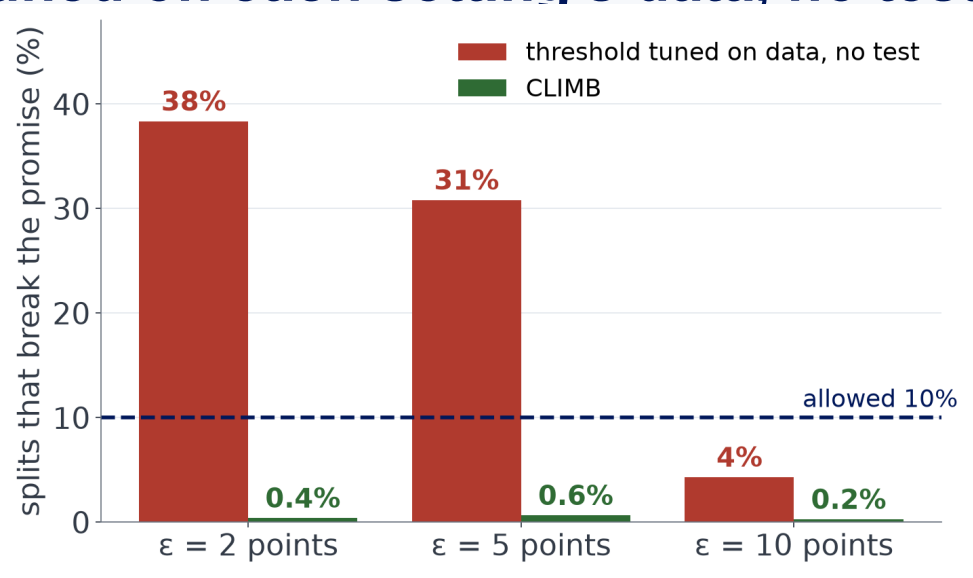
Two shortcuts, and both break the promise

One threshold for all settings



Above ε on **4 of 12 settings, up to 16 points.**

Tuned on each setting's data, no test



Breaks the promise on **31% of splits** at ε = 5.

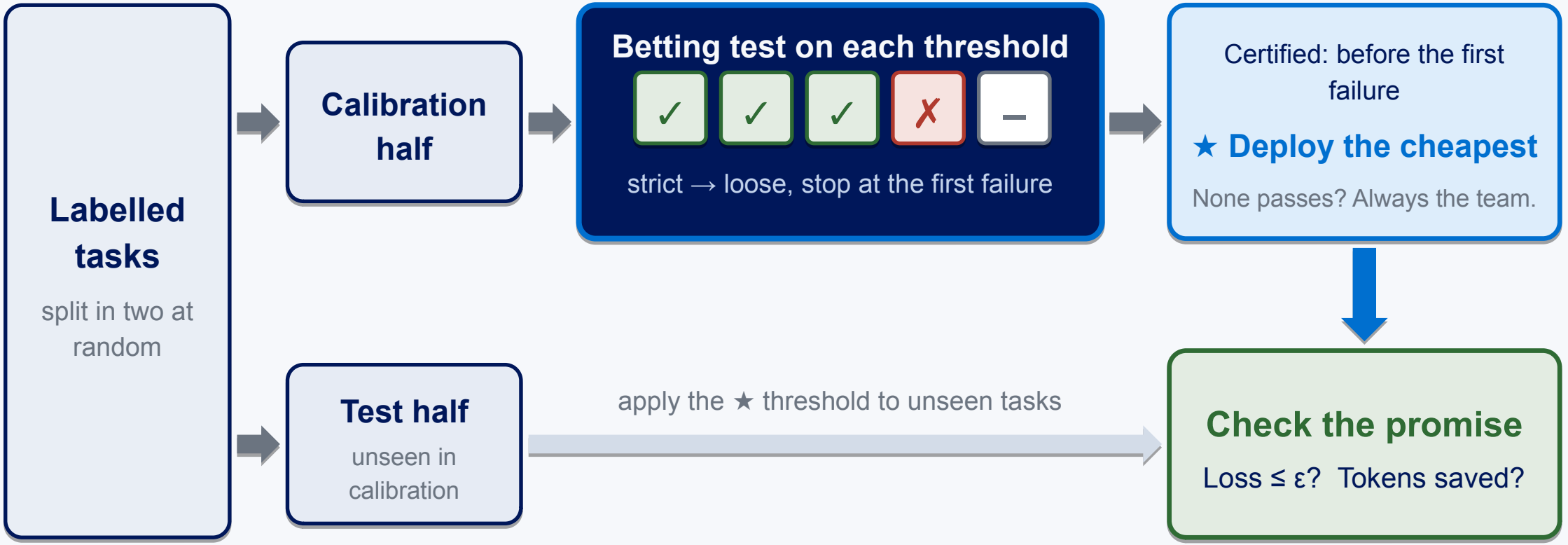
The right threshold depends on the model and task.

We need it set from each setting's data, with a guarantee.

STEP 2 · SETTING THE THRESHOLD

Replace guessing with a promise: calibrate, then test

You pick only ε, e.g. “lose at most 5 points”. The data picks the threshold.



In our experiments: 200 random splits. In deployment: only the calibration half, the test half is your future tasks.

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

Why a betting test?

We need: accept an unsafe threshold at most 5% of the time, from a few hundred tasks.

Saves count

A cheap answer can also save one the team breaks (up to 9%). The bet credits it.

Uses low variance

Most tasks lose 0. The bet size adapts to what it has seen.

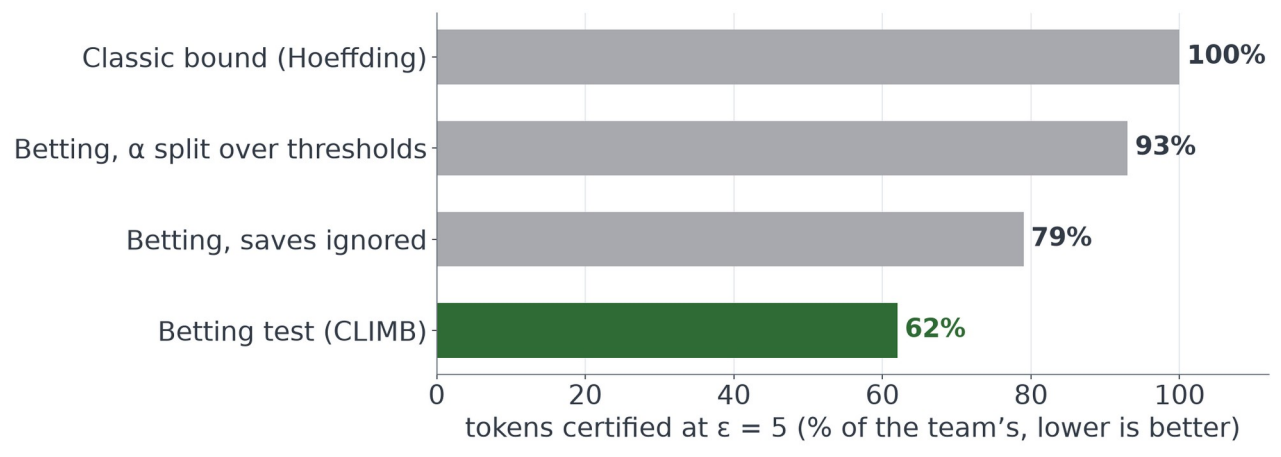
Many thresholds, full 5%

Tested strict → loose, each test keeps the whole 5%.

Any data

No model of the score. Only: calibration looks like deployment.

All valid. Which one saves the most?



62% vs 79–100%

of the team's tokens at ε = 5

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

The betting test: start with $1, bet the threshold is safe

Reveal the labelled tasks one by one. Each task multiplies the money:

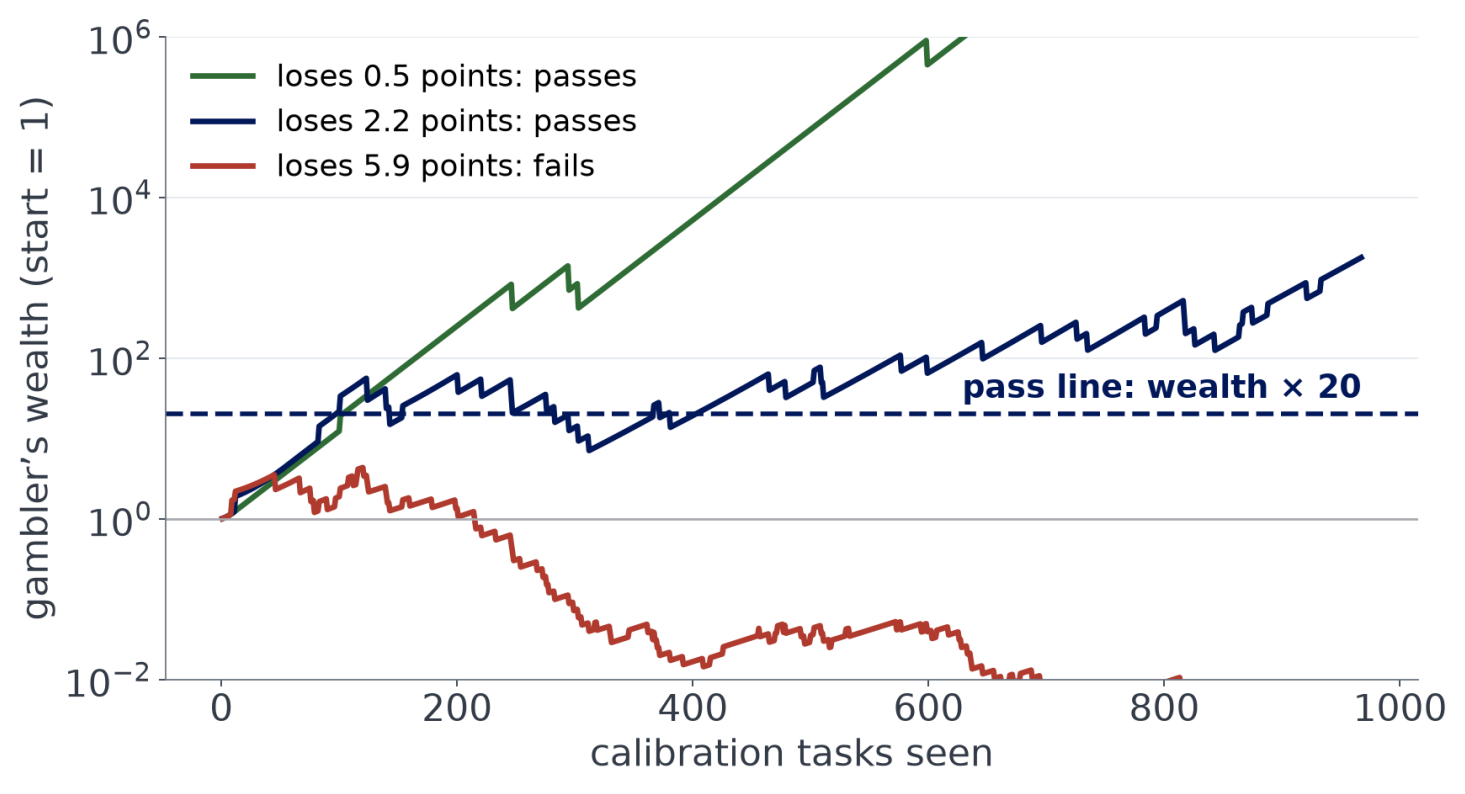
Cheap wrong, team right	Same result	Cheap right, team wrong
× 0.53	× 1.03	× 1.53
lose a lot	win a little (most tasks)	win a lot

Money ever reaches $20 → pass. Never reaches it → fail.

Multiplier = 1 + λ (ε − loss), loss = 1, 0 or −1. Shown for ε = 5 points and bet size λ = 0.5.

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

Safe thresholds get rich, unsafe ones go broke



Average multiplier

$$1 + \lambda(\varepsilon - \mu)$$

μ: the threshold's true average loss

Safe: μ < ε

average > 1: money grows, passes

Unsafe: μ > ε

average < 1: money shrinks, fails

Bet size λ: set from earlier tasks only.

Why $20? An unsafe threshold rarely gets there

Unsafe threshold

each bet loses on average

→

Average ≤ $1

starts at $1, never grows on average

→

Never below $0

a lost answer at most halves it

$$1 \geq \text{average wealth} \geq 20 \times P(\text{reach \$20})$$

so $P(\text{an unsafe threshold passes}) \leq 1/20 = 5\%$

100 worlds, $100 in total

Each pass holds $20, so at most 5 of 100 pass. $100 line → 1%.

A pass: accept, wrong ≤ 5% of the time

Like a driving test, not proof you can drive.

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

In CLIMB: testing one threshold, task by task

Task	Score > t?	Our answer	Team	Loss ℓ	× money	Money
1	yes, stop	✓	✓	**0**	× 1.03	**$1.03**
2	no → team	team's	✓	**0**	× 1.03	**$1.05**
3	yes, stop	✗	✓	**+1 lost**	× 0.53	**$0.55**
4	yes, stop	✓	✗	**−1 saved**	× 1.53	**$0.84**
5	yes, stop	✓	✓	**0**	× 1.03	**$0.86**
…						**≥ $20?**

ε = 5 points, bet size λ = 0.5. Escalated tasks use the team's answer: loss 0.

Each threshold, strict → loose

1 Tasks in random order, $1.

2 ℓ = team right − ours right.

3 λ from **earlier tasks only**. A loss at most halves the money.

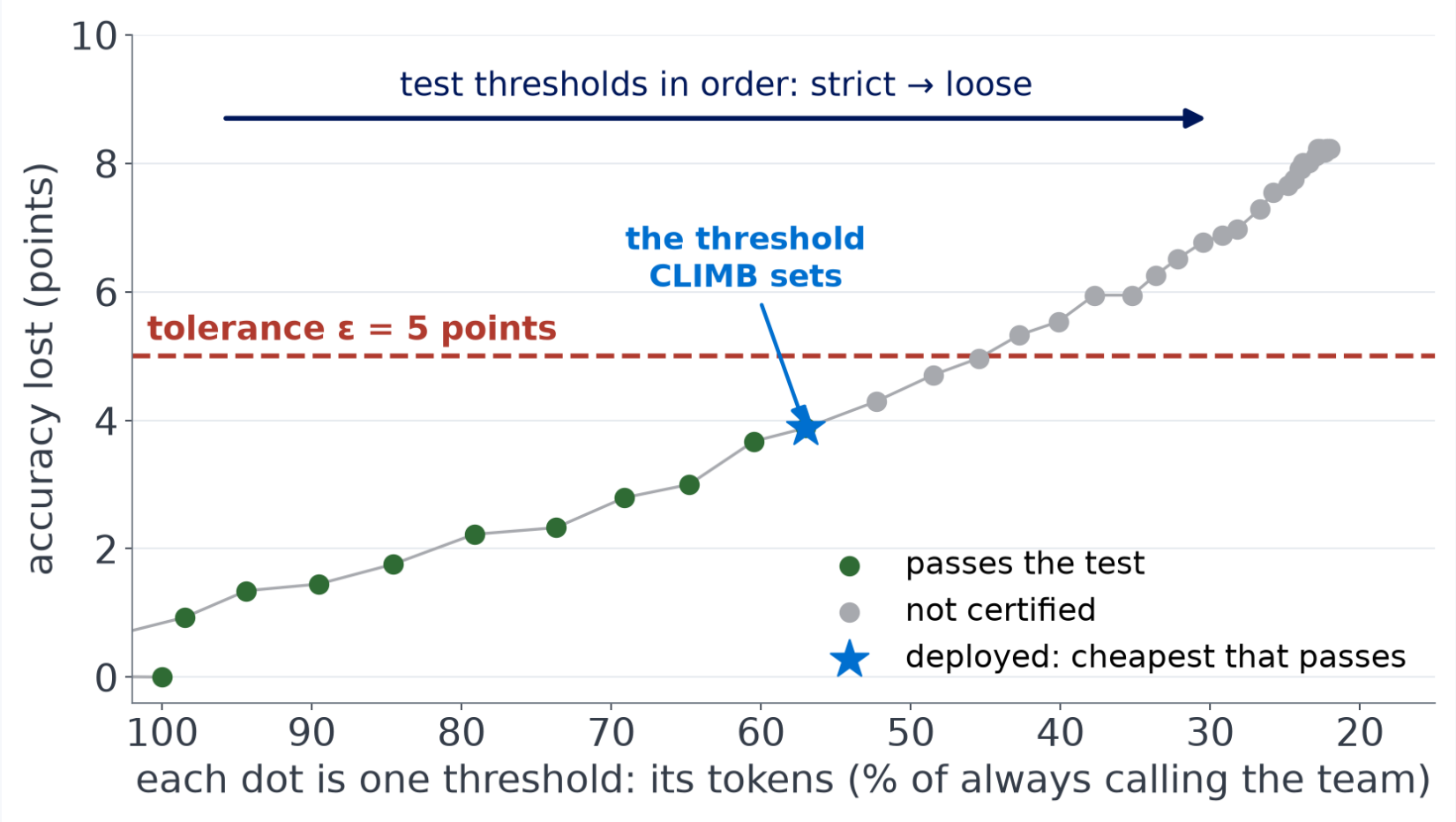
4 Money × (1 + λ (ε − ℓ)).

5 Ever ≥ $20: certified, next threshold. Never: stop.

Options A and B are each tested this way at 5%, so both hold together with probability ≥ 90%.

STEP 2 · SETTING THE THRESHOLD

Putting it together: how CLIMB sets the threshold

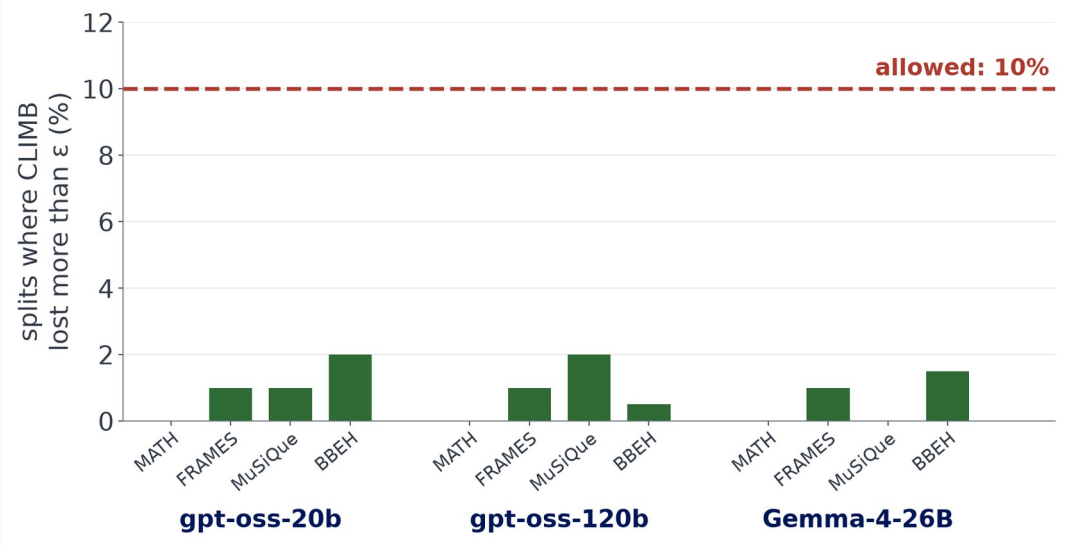


gpt-oss-120b on MuSiQue, threshold on the direct answer's confidence, one calibration half. Two-rung thresholds are tested the same way, the cheaper is kept.

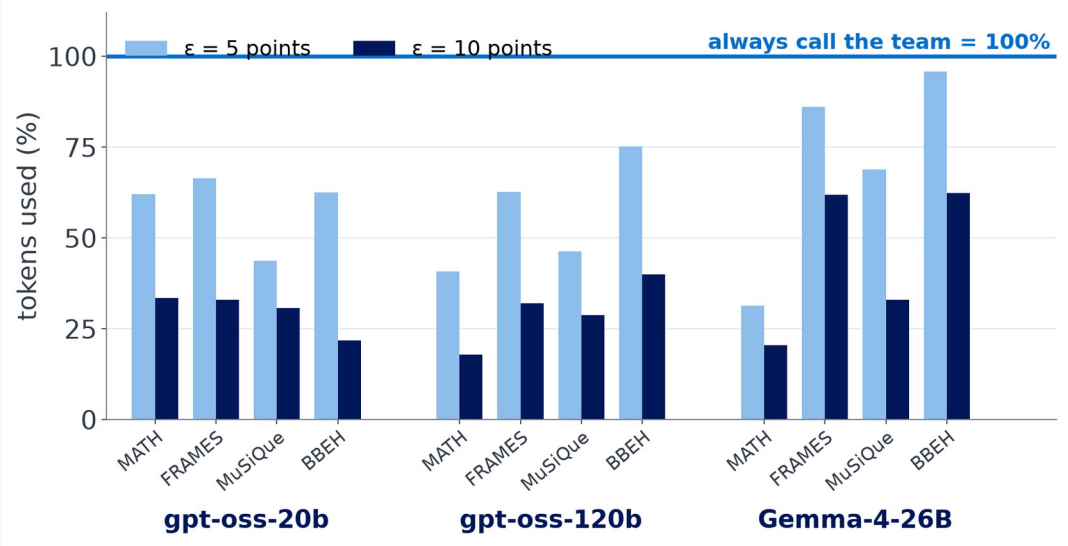
RESULTS · 3 MODELS × 4 BENCHMARKS, 200 RANDOM SPLITS

These thresholds keep the promise and save 38–65%

Splits that broke the promise (%)



Tokens used (% of the team's)



≤ 2%

of splits broke the promise. Allowed: 10%.

38% / 65% saved

allowing 5 / 10 points of loss

TAKEAWAYS

Call the team only when you must

Teams are costly

and sometimes break right answers.

Data-set threshold

you set the loss, a betting test finds the cheapest safe threshold.

38–65% saved

at 5 and 10 points, with the promise kept.

Assumes calibration tasks resemble deployment tasks. Recalibrating takes under a second.

FOR AMEX

What this offers Amex

1 Fewer tokens, even on closed models

Closed model writes the output → **Open model** reads it → log-probs → **CLIMB** uses them as S_0

Same idea as the reference model in PPO.

- Feed the closed model's output to an open model to get its **log-probs**
- **38–65% fewer tokens** at 5–10 points in our tests
- Agreement of two cheap calls needs **no log-probs**
- A weaker score saves less, the **promise still holds**

Getting a closed model's log-probs from an open model is a proposal, not yet tested.

2 Lessons for complex agent systems

More agents is not always better
Reviewers pass 23–58% of wrong drafts.

Route each task, not the whole system
Escalate only the doubtful tasks.

Cheap signals are strong
Two cheap answers agree → usually right.

Put a guarantee on any routing rule
Cascades, tool calls, human review.

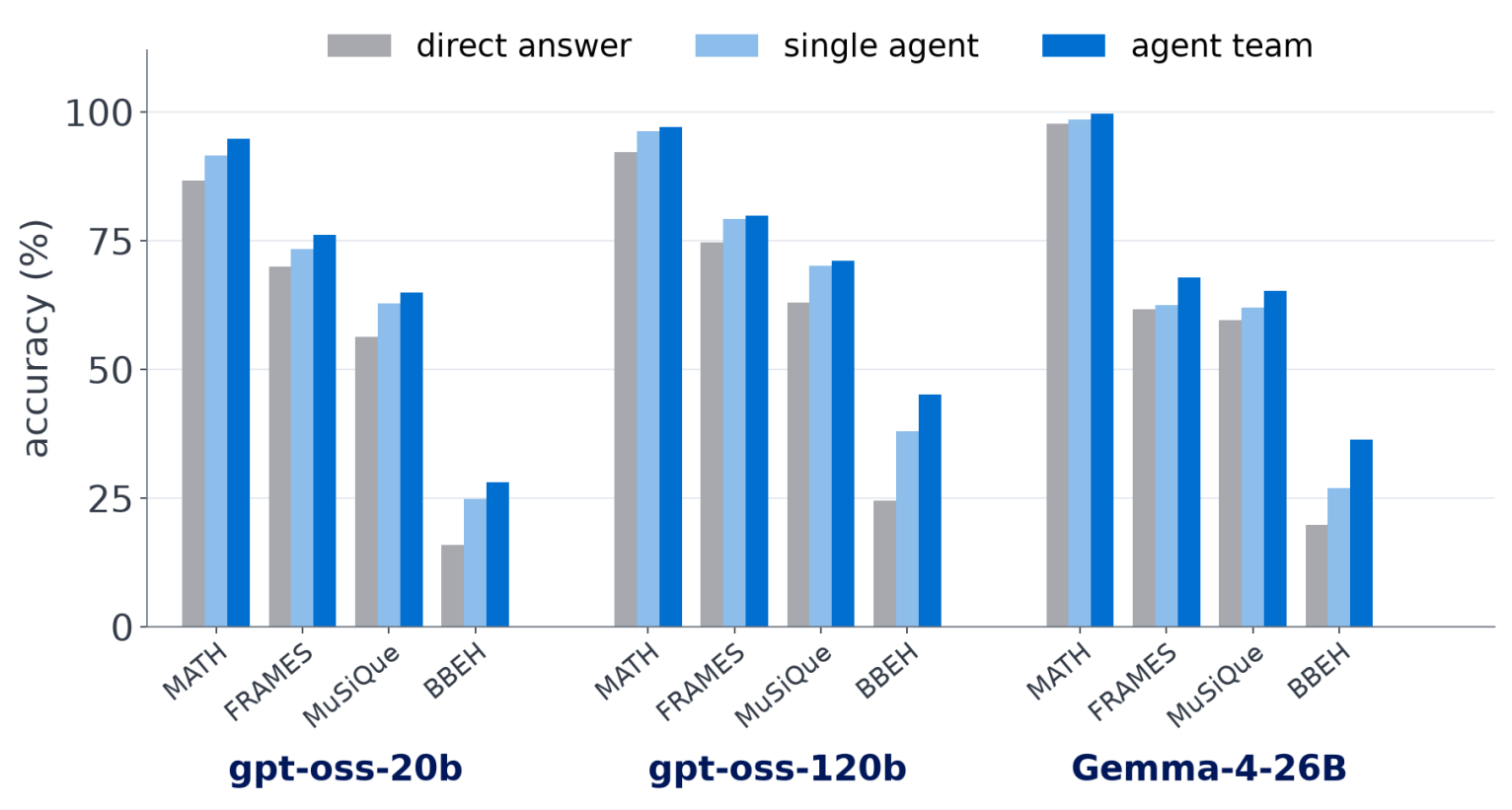
Thank you. Questions?

Backup

Details for questions

BACKGROUND

Teams are more accurate



+2 to +21

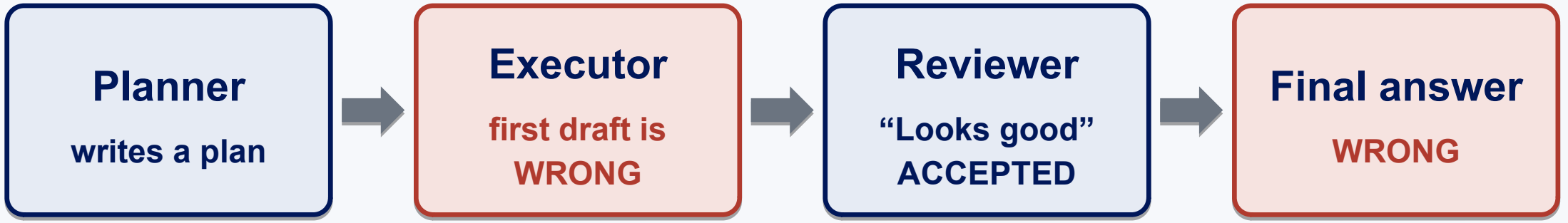
points over a direct answer

12 / 12

settings where the team is best, up to 9 points over a single agent

THE PROBLEM

Why does the team break right answers?



84%

of broken answers start as a wrong first draft the reviewer did not catch

23–58%

of wrong answers are accepted by the reviewer

HOW TO CLIMB

Deciding when to climb: four questions

1

Signal

Does the cheap answer look right?

→ **confidence and agreement**

2

Rule

Where do we draw the line?

→ **a threshold on the score**

3

Check

Is that line safe?

→ **a betting test: the core**

4

Depth

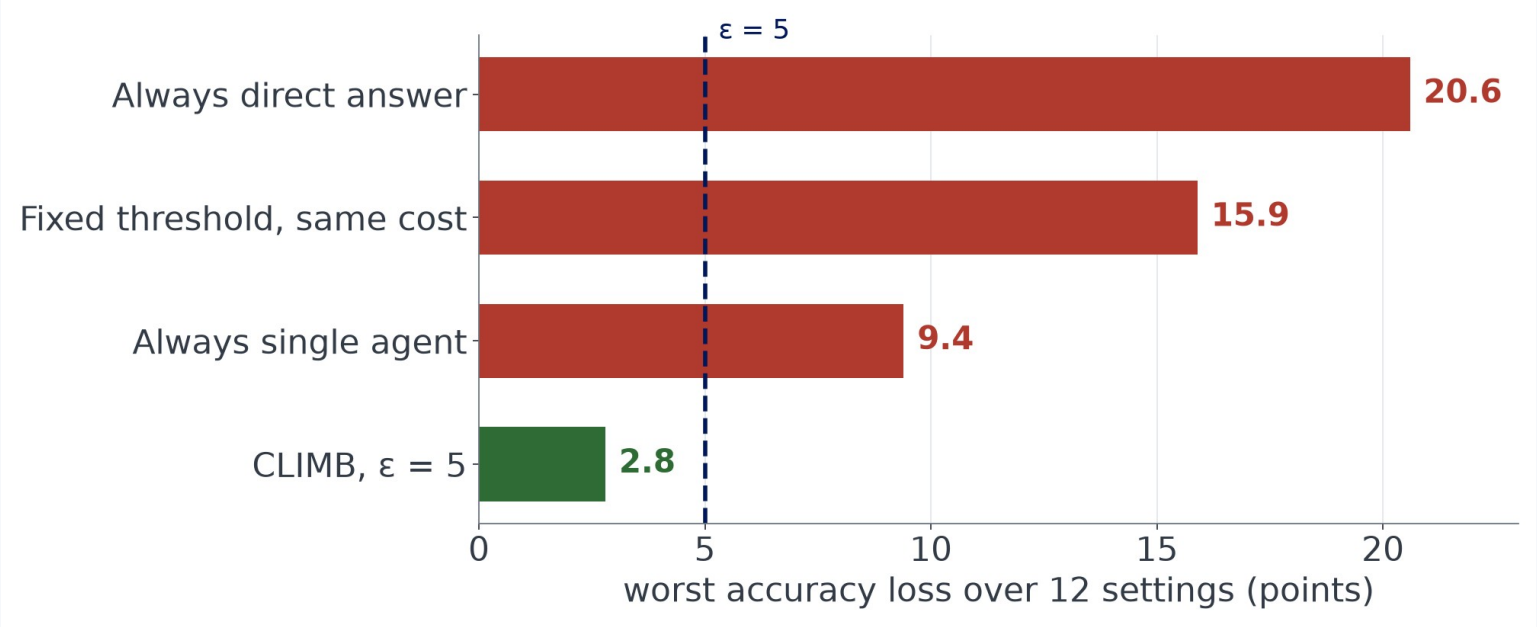
Look one rung up or two?

→ **let the data choose**

No rule passes the check? → always call the team. Worst case: no savings, never lost accuracy.

HOW TO CLIMB · RULE

Where to draw the line? Guessing can go badly



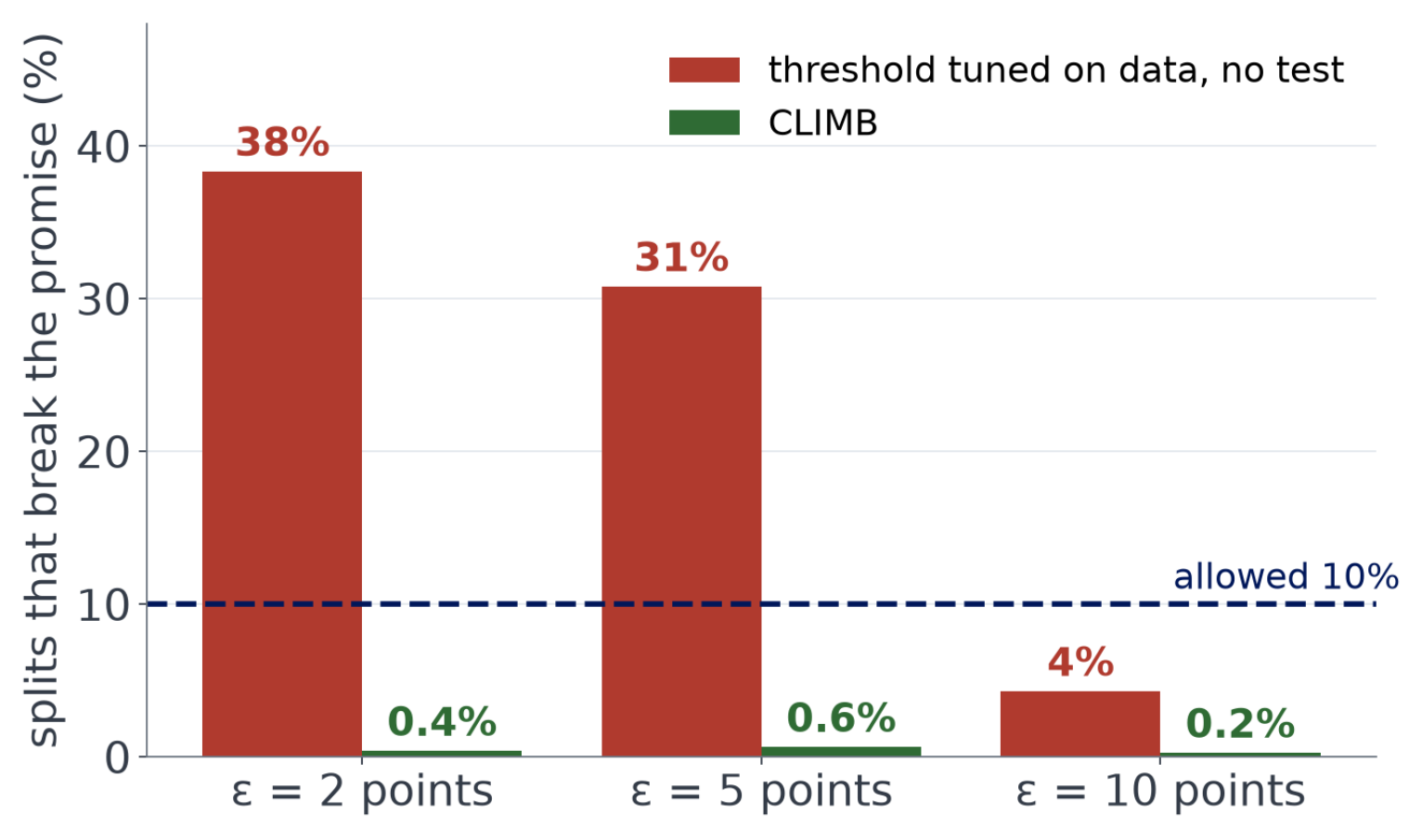
16 points

lost by a hand-picked threshold on one setting

Nothing at deployment says which setting will go wrong.

RESULTS

Without the test, the promise breaks



31%

of splits broken at ε = 5 by the tuned rule. CLIMB: 0.6%.

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

What a pass means, and what it does not

A bad rule reaches $20 at most 5% of the time.

So: reach $20 → accept the rule. This is wrong at most 5% of the time.

Right

"We accept it, and are wrong at most 5% of the time."

Wrong

"Reaching $20 proves the rule is good."

Wrong

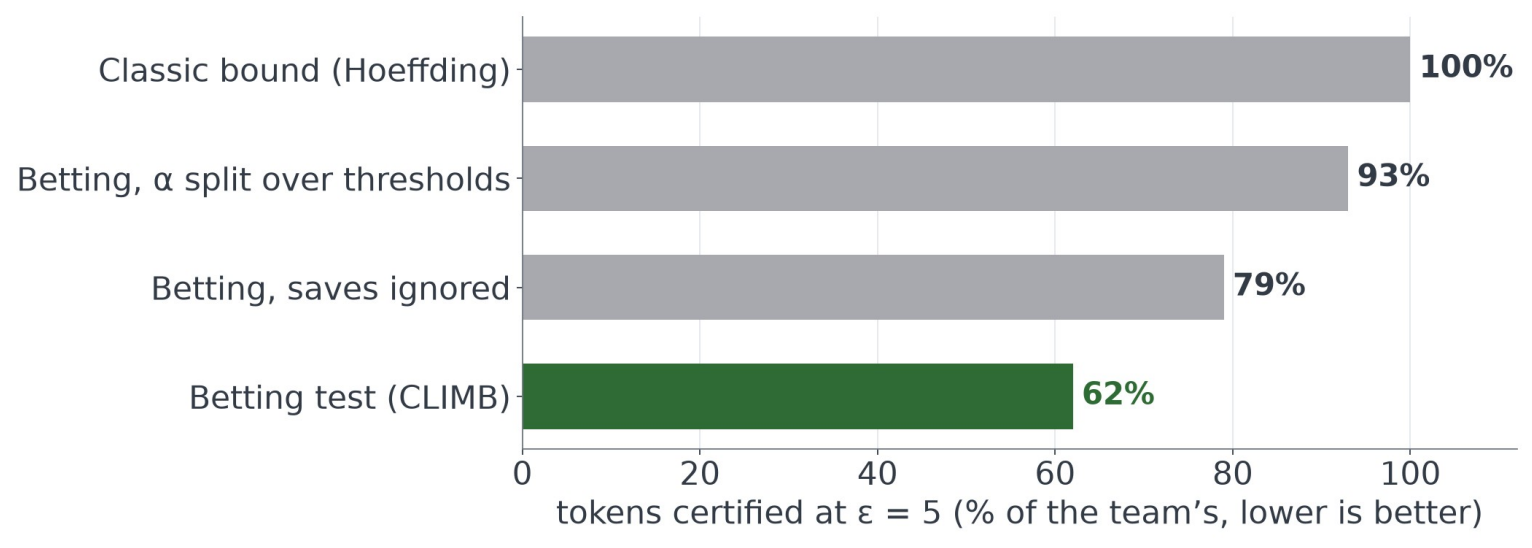
"5% of the rules that pass are bad."

Like a driving test. Bad drivers pass at most 5% of the time. Passing gets you a licence, it does not prove you can drive.

Two families, 5% each → the promise holds with probability ≥ 90%.

STEP 2 · SETTING THE THRESHOLD · THE BETTING TEST

Why bet on saved answers too? It saves more



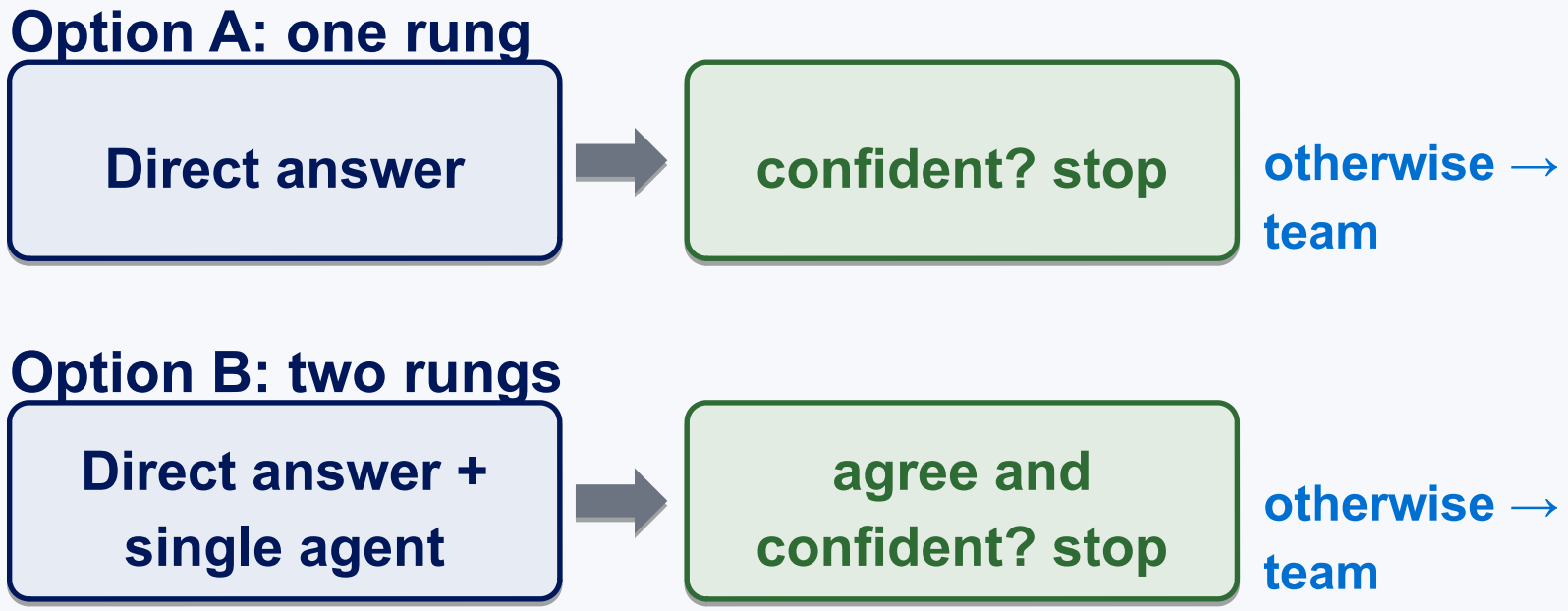
62%

of the team's tokens when saves count. Losses only: 79%.

Teams break right answers on up to 9% of tasks. The bet gives the rule credit for them.

Tokens each valid test can certify at ε = 5. Lower is better.

One rung or two? The data decides



Check both

on calibration data, then deploy the cheaper one that passes.

Neither passes? Always call the team.

EXPERIMENTS

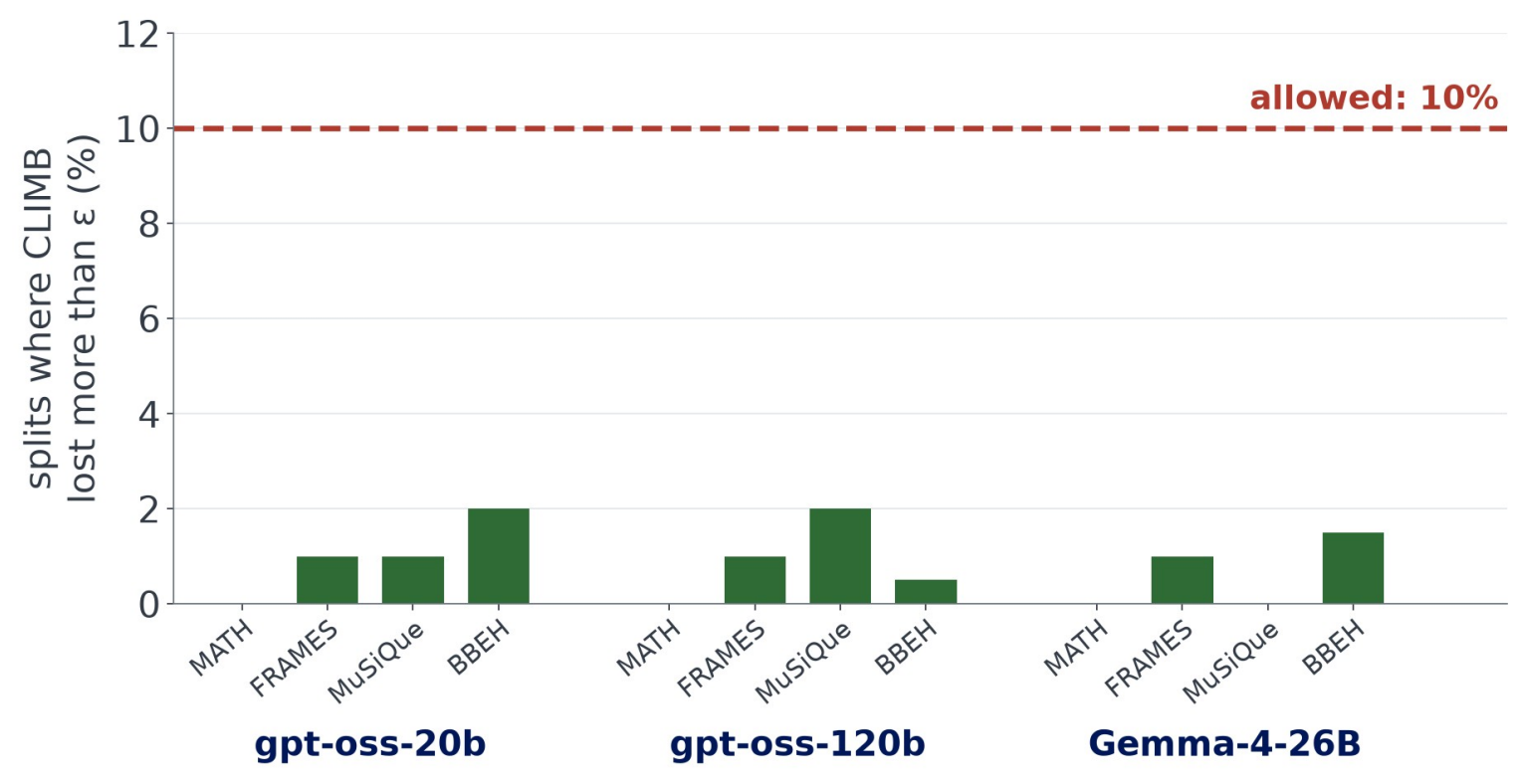
3 models × 4 benchmarks = 12 settings

	MATH math	FRAMES open-book QA	MuSiQue open-book QA	BBEH hard reasoning
gpt-oss-20b	**94.8%** team accuracy	**76.1%** team accuracy	**64.9%** team accuracy	**28.0%** team accuracy
gpt-oss-120b	**97.0%** team accuracy	**79.8%** team accuracy	**71.1%** team accuracy	**45.1%** team accuracy
Gemma-4-26B	**99.8%** team accuracy	**67.9%** team accuracy	**65.3%** team accuracy	**36.4%** team accuracy

Record every task once, then replay **200 random splits**: half to calibrate CLIMB, half to test it.

RESULTS

The promise holds on every setting

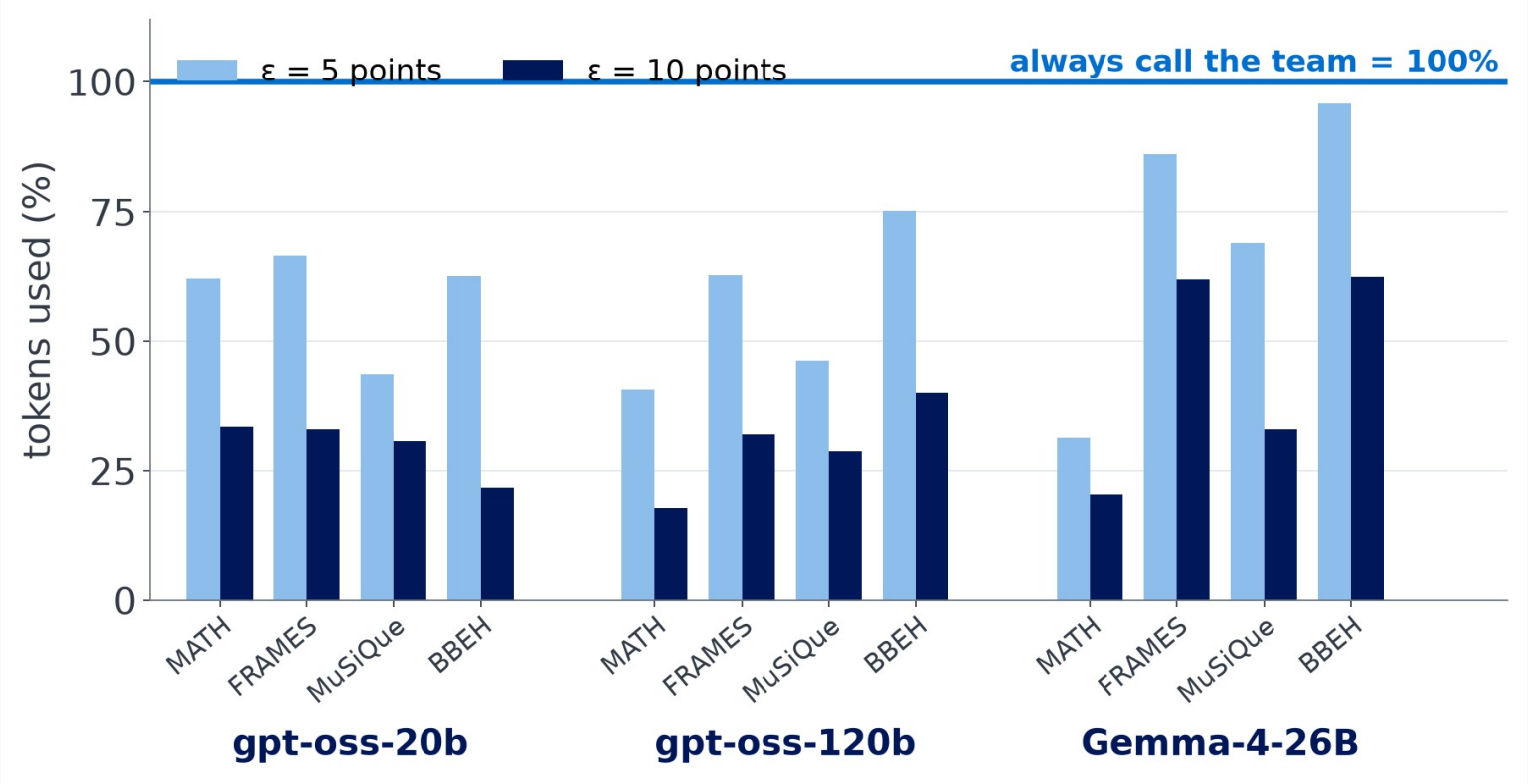


≤ 2%

of splits broke the promise. Allowed: 10%.

RESULTS

How much it saves



38% saved

allowing 5 points of loss

65% saved

allowing 10 points of loss